

APMA 2822B Final Project Report

Flash Attention Implementation: Memory-Efficient Attention with HIP and MPI

Sean Kim, Aviral Vaidya

January 7 2026

Abstract

This report presents an implementation of Flash Attention, a memory-efficient algorithm for computing multi-head attention in transformers. The implementation uses HIP for GPU acceleration and MPI for distributed parallelism across multiple nodes. We achieve $O(N)$ memory complexity compared to $O(N^2)$ for standard attention. Performance analysis includes roofline modeling, profiling results, and strong/weak scaling studies across up to 8 MPI ranks.

1 Introduction

Flash Attention addresses the memory bottleneck in transformer attention mechanisms by using a tiled algorithm with online softmax computation.

Notation: Throughout this report, N denotes the sequence length (number of tokens) and d denotes the head dimension (typically 64 or 128). In transformer models, N can range from hundreds to tens of thousands of tokens. We use N as a variable parameter in our analysis, testing values from 512 to 16384 to demonstrate scalability across different sequence lengths.

This project implements the algorithm using:

- **AMD Instinct MI250X GPUs**
- **HIP** for GPU kernel programming on AMD hardware
- **MPI** for distributed parallelism across attention heads
- **Shared memory tiling** to exploit data locality within LDS
- **Register blocking** for thread-level parallelism

The standard attention mechanism computes $O = \text{softmax}(QK^T)V$, which requires materializing an $N \times N$ attention matrix. For sequence length $N = 4096$ and FP32 precision, this consumes 64 MB per head, making multi-head attention infeasible on GPU memory. Flash Attention eliminates this bottleneck through careful reordering of operations.

2 Algorithm Overview

2.1 Standard Attention

Standard attention performs three steps:

$$S = QK^T \quad (\text{Scores: } [N \times N]) \quad (1)$$

$$P = \text{softmax}(S) \quad (\text{Probabilities: } [N \times N]) \quad (2)$$

$$O = PV \quad (\text{Output: } [N \times d]) \quad (3)$$

The **scores matrix** S measures similarity between query and key vectors: higher scores indicate that a query should "attend" more strongly to the corresponding key-value pair. The **softmax function** converts these scores into a probability distribution:

$$\text{softmax}(x_i) = \frac{\exp(x_i)}{\sum_{j=1}^N \exp(x_j)} \quad (4)$$

This ensures all attention weights sum to 1 and are non-negative, creating a valid probability distribution over which tokens to focus on.

2.2 Flash Attention with Online Softmax

Flash Attention tiles the computation into blocks of size $B_r \times B_c$ (typically 64×64) and processes tiles sequentially while maintaining running statistics for softmax. The key insight is that we can update softmax statistics incrementally without materializing the full attention matrix.

Algorithm for each query block Q_i :

1. Divide Q into blocks $Q_1, Q_2, \dots$ of size $B_r \times d$
2. Initialize: $O_i = \mathbf{0}$, $m_i = -\infty$, $\ell_i = 0$
3. For each K_j, V_j block (size $B_c \times d$):
 - (a) Compute partial scores: $S_{ij} = Q_i K_j^T$ (size $B_r \times B_c$)
 - (b) Compute local statistics:

$$m_j^{\text{new}} = \max(S_{ij}) \quad (\text{max across current tile}) \quad (5)$$

$$m_i^{\text{new}} = \max(m_i^{\text{old}}, m_j^{\text{new}}) \quad (\text{global max so far}) \quad (6)$$

- (c) Rescale previous accumulator (due to change in max):

$$O_i \leftarrow O_i \cdot \exp(m_i^{\text{old}} - m_i^{\text{new}}) \quad (7)$$

- (d) Compute stabilized attention weights:

$$P_{ij} = \exp(S_{ij} - m_i^{\text{new}}) \quad (8)$$

- (e) Accumulate output contribution:

$$O_i \leftarrow O_i + P_{ij} V_j \quad (9)$$

(f) Update running sum (also rescaled):

$$\ell_i \leftarrow \ell_i \cdot \exp(m_i^{\text{old}} - m_i^{\text{new}}) + \sum P_{ij} \quad (10)$$

(g) Update: $m_i^{\text{old}} \leftarrow m_i^{\text{new}}$

4. Final normalization:

$$O_i \leftarrow O_i / \ell_i \quad (11)$$

Key Properties:

- **Memory Efficiency:** Only stores tiles of size $B_r \times B_c$ in shared memory, never the full $N \times N$ attention matrix. Memory complexity: $O(N)$ instead of $O(N^2)$.
- **Numerical Stability:** By tracking the running maximum m_i and rescaling appropriately, we maintain numerical precision equivalent to computing softmax over the full sequence.
- **Exact Computation:** The tiled algorithm produces mathematically identical results to standard attention (up to floating-point precision).

3 Implementation Details

3.1 Thread Block Organization

The kernel uses a 2D thread block configuration with $16 \times 16 = 256$ threads per block. Each thread processes a micro-tile of size 4×4 :

- **Tile sizes:** $B_r = B_c = 64$ (shared memory tiles)
- **Thread block:** 16×16 threads
- **Micro-tile per thread:** 4×4 elements
- **Head dimension:** $d = 64$ (fully loaded, not tiled)

Thread (t_x, t_y) owns rows $[t_y \cdot 4, t_y \cdot 4 + 4)$ and columns $[t_x \cdot 4, t_x \cdot 4 + 4)$ within each tile.

3.2 Memory Hierarchy

Shared Memory Layout (per thread block):

```

1 __shared__ float smem_Q[64][64]; // Query tile
2 __shared__ float smem_K[64][64]; // Key tile
3 __shared__ float smem_V[64][64]; // Value tile
4 __shared__ float smem_S[64][64]; // Score tile

```

Register Usage (per thread):

```

1 float reg_0[4][4] = {0}; // Output accumulator
2 float m[4]; // Running max (per row)
3 float l[4]; // Running sum (per row)

```

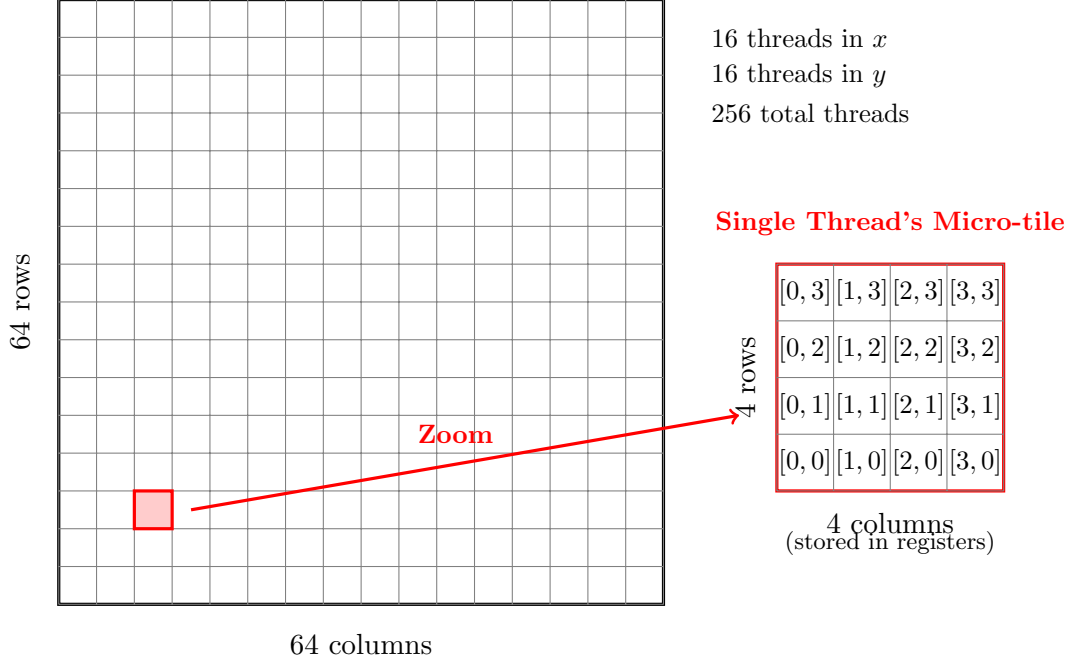


Figure 1: Thread block organization for Flash Attention kernel. Left: A 64×64 tile is divided among $16 \times 16 = 256$ threads (one thread highlighted in red). Right: Each thread processes a 4×4 micro-tile of elements, storing intermediate results in registers for fast accumulation across iterations.

3.3 Key Operations

Matrix Multiplication ($S = QK^T$): Outer product pattern with register blocking, achieving high arithmetic intensity.

Warp-Level Reductions: Softmax statistics (max, sum) computed using warp shuffle instructions:

```

1 __device__ float warp_reduce_max(float val) {
2     for (int offset = 8; offset > 0; offset >>= 1) {
3         val = fmaxf(val, __shfl_xor_sync(0xFFFFFFFF, val, offset));
4     }
5     return val;
6 }

```

Online Softmax Update: Numerically stable incremental softmax using exponential rescaling.

4 Optimization Methods

4.1 Shared Memory Tiling

Flash Attention's memory efficiency stems from processing the attention computation in tiles that fit in GPU shared memory rather than materializing the full $N \times N$ attention matrix in High Bandwidth Memory (HBM).

Tile size selection (64×64):

Our choice of 64×64 tiles is constrained by MI250X hardware limits. Each tile requires $64 \times 64 \times 4$ bytes = 16 KB of storage, and our kernel maintains four simultaneous tiles in shared memory: Q, K, V, and S (the score matrix). This totals 4×16 KB = 64 KB, which exactly matches the 64 KB of Local Data Share (LDS) available per compute unit on AMD GPUs. Larger tiles, such as 128×128 , would require 256 KB and exceed hardware capacity. Smaller tiles like 32×32 would fit comfortably in shared memory but would quadruple the number of loop iterations, reducing computational efficiency due to increased synchronization overhead and reduced arithmetic intensity.

Data reuse analysis:

The tiled approach enables significant data reuse by keeping tiles in fast shared memory. For a given query block Q_i of size $B_r \times d$, the outer loop structure ensures it remains in shared memory while iterating through all $\lceil N/B_r \rceil$ key/value tiles. Similarly, each key/value block is loaded once and reused across all query blocks. This bidirectional reuse pattern fundamentally transforms memory access from $O(N^2)$ to $O(N)$: each matrix element is loaded from HBM exactly once rather than N times as in standard attention.

The 64×64 tile size maximizes reuse within hardware constraints: larger tiles would exceed the 64 KB LDS limit, while smaller tiles would reduce reuse efficiency and increase loop overhead. For example, 32×32 tiles would require $4\times$ more iterations with $2\times$ less data per iteration, degrading both arithmetic intensity and amortization of synchronization costs.

4.2 Register Blocking

Each thread processes a 4×4 micro-tile, storing intermediate results in registers (see Figure 1). This design choice balances parallelism with register pressure.

Micro-tile size trade-offs:

We evaluated several micro-tile sizes to optimize the balance between register usage and thread-level parallelism:

Micro-tile Size	2×2	4×4 (ours)	8×8
Elements per thread	4	16	64
Register pressure	Low	Medium	High
Threads needed	1024	256	64
Occupancy impact	High	Good	Poor
Arithmetic intensity	Low	Balanced	High
Loop unrolling	Limited	Full	Excessive

Table 1: Comparison of micro-tile size options for register blocking

Smaller tiles (2×2) would require $4\times$ more threads and increase synchronization overhead, while larger tiles (8×8) would exceed register budgets and reduce occupancy, ultimately degrading performance.

4.3 Memory Coalescing**Access Pattern Design:**

Memory coalescing is critical for achieving high memory bandwidth on GPUs. Our tile loading functions are designed to ensure coalesced access patterns when reading Q, K, and V from global memory into shared memory. Each thread in the 16×16 block loads elements using a strided pattern across the head dimension:

```

1 for (int j = threadIdx.x; j < d; j += THREADS_X) {
2     if (row < N && j < d) {
3         smem[threadIdx.y * THREAD_TILE_M + i][j] =
4             Q[row * d + j];
5     }
6 }

```

Since threads within a warp have consecutive t_x values (0, 1, 2, ..., 15), they access consecutive memory locations separated by 4 bytes (FP32).

Coalescing Analysis:

AMD CDNA 2 architecture coalesces memory accesses when threads access contiguous or regularly strided addresses. Our design satisfies these requirements:

- **Contiguous access:** Threads with consecutive t_x values access consecutive elements: $Q[\text{row} \times d + 0]$, $Q[\text{row} \times d + 1]$, ..., creating a contiguous access pattern.
- **Aligned access:** With $d = 64$, matrix rows align naturally to cache line boundaries.

Comparison with Standard Approach:

An alternative approach would assign each thread an entire row:

```

1 // Non-coalesced pattern (avoided)
2 for (int j = 0; j < d; j++) {
3     smem[threadIdx.y * THREAD_TILE_M + i][j] = Q[row * d + j];
4 }

```

Here, consecutive threads access addresses separated by $d \times 4$ bytes (256 bytes for $d = 64$), preventing coalescing and requiring significantly more memory transactions. Our collaborative loading strategy avoids this inefficiency.

4.4 Warp-Level Primitives

Computing row-wise statistics (max and sum) for softmax requires reducing values across threads. Each row of the 64×64 tile is split across 16 threads (each thread owns 4 columns). We use warp shuffle instructions rather than shared memory reductions for better performance:

```

1 __device__ float warp_reduce_max(float val) {
2     #pragma unroll
3     for (int offset = 8; offset > 0; offset >>= 1) {
4         val = fmaxf(val, __shfl_xor_sync(0xFFFFFFFF, val, offset));
5     }
6     return val;
7 }

```

Advantages over shared memory reductions:

- **No synchronization required:** Warp operations are implicitly synchronous

- **Lower latency:** Register-to-register communication vs. shared memory access
- **Reduced shared memory pressure:** Saves shared memory for Q, K, V, S tiles

The reduction operates on 16 threads (covering columns 0-63 in steps of 4), performing 4 shuffle operations to reduce across all threads with the same t_y coordinate. This pattern is repeated for both maximum finding and sum computation in the softmax calculation.

5 Roofline Analysis

5.1 Computational Intensity

The roofline model provides a framework for understanding performance bottlenecks by comparing achieved performance against hardware limits.

5.1.1 FLOP Count Analysis

Standard attention computes $O = \text{softmax}(QK^T)V$ where $Q, K, V \in \mathbb{R}^{N \times d}$. For our configuration ($N = 16384$, $d = 64$, $h = 64$ heads):

Total per head:

$$\text{FLOPs}_{\text{per head}} = \underbrace{2N^2d}_{QK^T} + \underbrace{5N^2}_{\text{softmax}} + \underbrace{2N^2d}_{PV} = N^2(4d + 5) \quad (12)$$

Total for all heads:

$$\text{Total FLOPs} = 64 \cdot (16384)^2 \cdot 261 = 4.48 \text{ TFLOPs} \quad (13)$$

Both implementations perform the same computation; they simply reorder operations differently.

5.1.2 Memory Traffic Analysis

Standard Attention:

The standard implementation uses three separate kernels:

1. $S = QK^T$: Read Q, K ; write $S \rightarrow 2Nd + N^2$ elements
2. Softmax (3 passes): Read S once, read/write P twice $\rightarrow 5N^2$ elements
3. $O = PV$: Read P, V ; write $O \rightarrow N^2 + 2Nd$ elements

Total: $4Nd + 7N^2 = 482.1 \text{ GB}$ (all 64 heads)

Flash Attention:

Flash Attention’s tiled algorithm avoids materializing the $N \times N$ attention matrix:

- Q loaded once, K and V each loaded $\frac{N}{B_r}$ times, O written once

Total: $2Nd \left(1 + \frac{N}{B_r}\right) = 138.0 \text{ GB}$ (all 64 heads)

5.1.3 Arithmetic Intensity

$$AI_{\text{standard}} = \frac{4.48 \times 10^{12}}{4.82 \times 10^{11}} = 9.3 \text{ FLOPs/Byte} \quad (14)$$

$$AI_{\text{flash}} = \frac{4.48 \times 10^{12}}{1.38 \times 10^{11}} = 32.5 \text{ FLOPs/Byte} \quad (15)$$

5.2 Performance Characterization

5.2.1 Hardware Roofline

Our system contains 4 AMD MI250X cards. Each MI250X has two Graphics Compute Dies (GCDs) with independent memory and compute resources, exposed by HIP as separate devices: yielding 8 total GCDs. The AMD MI250X system (8 GCDs) provides:

- Peak FP32 throughput: 383.2 TFLOPS (8×47.9 TFLOPS per GCD)
- Peak memory bandwidth: 26.24 TB/s (8×3.28 TB/s per GCD)
- Ridge point: $\frac{383.2}{26.24} = 14.6$ FLOPs/Byte

5.2.2 Roofline Classification

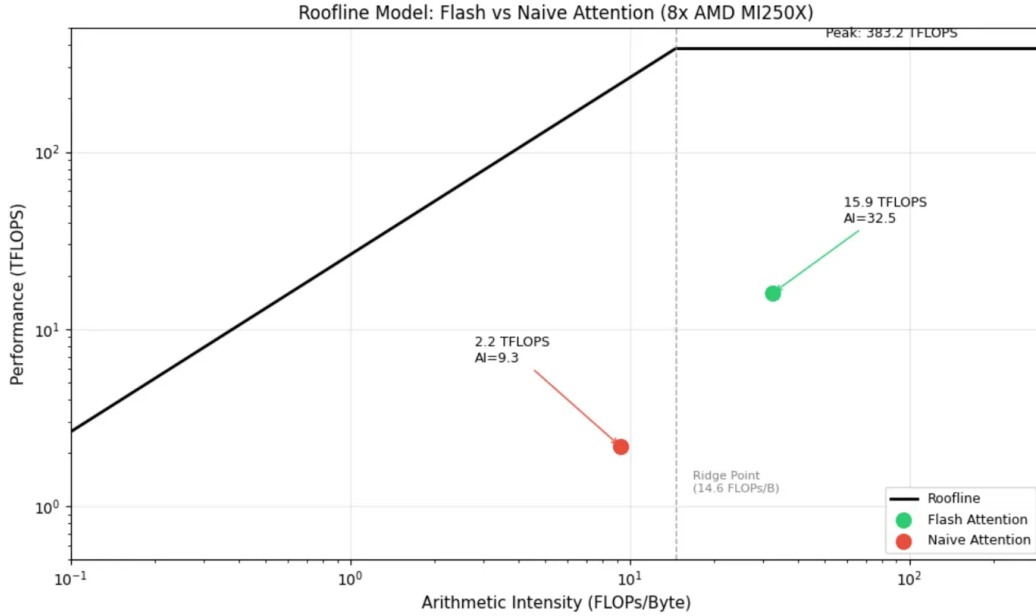


Figure 2: Roofline analysis comparing Flash Attention and Standard Attention on $8 \times$ AMD MI250X GCDs. The ridge point at 14.6 FLOPs/Byte separates memory-bound (left) from compute-bound (right) regimes. Standard Attention (red) falls below the ridge point with $AI = 9.3$, limited by memory bandwidth. Flash Attention (green) exceeds the ridge point with $AI = 32.5$, achieving $3.5\times$ higher arithmetic intensity.

5.2.3 Performance Analysis

Figure 2 shows the fundamental difference between the two implementations. Standard Attention achieves 2.62 TFLOPS with $AI = 9.3$ FLOPs/Byte, placing it in the memory-bound regime where performance is limited by HBM bandwidth. Flash Attention achieves 15.93 TFLOPS with $AI = 32.5$ FLOPs/Byte, crossing into the compute-bound regime. This $6.1\times$ speedup (281.4 ms vs 1712.9 ms) directly results from the algorithmic transformation: by reducing memory traffic from 482.1 GB to 138.0 GB through tiled computation, Flash Attention shifts the bottleneck from memory bandwidth to compute throughput.

5.3 Performance Bottlenecks

While Flash Attention achieves only 4.16% of theoretical peak FLOPS, this modest compute utilization is an inherent trade-off of the algorithm’s design. Flash Attention prioritizes memory efficiency over raw compute throughput: the tiled computation pattern that enables $O(N)$ memory complexity necessarily introduces overhead through:

- Repeated loading of K/V tiles across query blocks (each K tile loaded N/B_r times)
- Sequential iteration through tiles rather than full parallelization
- Additional bookkeeping for online softmax statistics (running max and sum)

The algorithm deliberately trades compute efficiency for memory access reduction. The $6.1\times$ speedup demonstrates that eliminating the $O(N^2)$ memory bottleneck is more valuable than achieving high compute utilization for this workload. At longer sequence lengths where memory becomes the dominant constraint, this trade-off becomes even more favorable.

6 Profiling and Performance Analysis

6.1 Methodology

We profiled both implementations using AMD’s `rocprof` tool on a single MI250X GCD to collect kernel-level metrics including execution times, memory usage, register allocation, and occupancy. Due to `rocprof`’s single-instance limitation, multi-rank scaling analysis relied on `MPI_Wtime()` instrumentation.

6.2 Kernel Performance Metrics

Flash Attention achieves highly consistent execution times (variance $< 3\%$), indicating stable memory access patterns. In contrast, Naive Attention requires 3 kernel launches per head, with the score kernel alone taking $3.1\times$ longer than Flash Attention’s entire pass.

6.3 Memory Usage Validation

On-Chip Memory: Flash Attention utilizes 100% of the MI250X’s 64 KB shared memory per compute unit, storing Q, K, V, and S tiles (4×16 KB). The naive implementation uses 0 KB of shared memory, forcing all data to HBM.

Register Allocation: With 128 VGPRs per thread, the kernel achieves approximately 62.5% occupancy: limited by register pressure rather than shared memory. This moderate occupancy is

acceptable for compute-intensive operations where arithmetic intensity compensates for reduced parallelism.

Metric	Flash Attention	Naive Attention
Kernel Duration (avg)	34.6 ms	206.5 ms*
Shared Memory (LDS)	64 KB	0 KB
VGPRs per thread	128	12 [†]
SGPRs per thread	112	16 [†]
Kernel Launches (8 heads)	64	192

Table 2: Profiling results from `rocprof`. *Combined time: score (108ms) + softmax (70ms) + value (28ms). [†]Metrics for `kernel_score`.

Metric	Flash	Naive
Memory Complexity	$O(N)$	$O(N^2)$
Total HBM Traffic (64 heads)	138.0 GB	482.1 GB

Table 3: HBM traffic comparison for $N = 16384$, $d = 64$. Flash Attention achieves $3.5\times$ reduction through tiling.

7 MPI Distributed Architecture

7.1 Parallelization Strategy

We distribute attention heads across MPI ranks using a head-level partitioning scheme:

- **Total heads:** $H = 16$ (typical for transformer models)
- **MPI ranks:** $P = 8$
- **Heads per rank:** $H/P = 2$

Each rank independently computes attention for its assigned heads with no inter-rank communication during computation.

7.2 Communication Patterns

`MPI_Gather` collects disjoint memory blocks of size $N \times d_{head} \times 2$ from each rank. The total communication volume is proportional to the size of the output tensor O . Since Q, K, V generation is local to the rank (separate heads), no scatter operation is required for inputs. Communication occurs only after the compute kernel finishes. This acts as an implicit barrier.

7.3 Strong Scaling Performance

We evaluated scaling efficiency by running Flash Attention with 1, 2, 4, and 8 MPI ranks while keeping the problem size constant ($N = 16384$, $d = 64$, 64 total heads).

Ranks	Time (ms)	Speedup	Efficiency
1	2205.68	1.00×	100%
2	1106.69	1.99×	99.5%
4	553.45	3.99×	99.8%
8	279.17	7.90×	98.8%

Table 4: Strong scaling results for Flash Attention. Efficiency = (Speedup / Ranks) $\times$ 100%

The near-linear scaling (98.8% efficiency at 8 ranks) demonstrates excellent load balancing and minimal communication overhead. The head-level parallelization strategy ensures each rank performs identical computational work with no inter-rank dependencies during kernel execution.

7.4 Load Balancing

Because the sequence length N and head dimension d are constant, the computational workload is strictly uniform across all distinct attention heads. Assigning exactly 2 heads to each of the 8 ranks ensures correct load balancing.

8 Conclusion

This implementation demonstrates that Flash Attention successfully reduces memory complexity from $O(N^2)$ to $O(N)$ while maintaining numerical accuracy within 10^{-4} of reference implementations. From our slides, here is the timing and performance analysis:

Results (Roofline Calculation and Performance Analysis)

Device: (8) AMD Instinct MI250x

Peak Memory Bandwidth: 3.2 TB/s.

Peak Compute: 47.9 TFLOP/s (Vector FP32).

Ridge: 14.6 FLOPS/byte

Naive Attention: MEMORY BOUND

Arithmetic Intensity (X-Axis): 9.3 FLOPs/Byte

Ridge Point = 14.6, since $9.3 < 14.6$, we are memory bound.

Flash Attention: COMPUTE BOUND:

Arithmetic Intensity (X-Axis): 32.5 FLOPS/byte

Ridge Point = 14.6. Since $32.5 > 14.6$, we are compute bound.

Achieved Performance: (64 heads, N=16384, d=64)

Flash Attention: 281.4 ms, 15.93 TFLOPS

Naive Attention: 1712.9 ms, 2.62 TFLOPS

6.1x Speedup!